Cleanverse Compliance Protocol (CCP) CVA Integration Guide

> This document is intended for **integration developers** and explains how to issue **CVA (Cleanverse Verified Asset)** under the Cleanverse Compliance Protocol (CCP) via the **RuleV2** mechanism.
>
> Perspective: You need to issue a new compliant ERC20 token (stablecoin, RWA, etc.) directly on Cleanverse. This guide focuses on the field semantics, API usage, and contract templates of the **RuleV2** compliance policy. Legacy Rule details are hidden from developers.
>
> Supported networks: EVM (Ethereum, Base, BSC, Arbitrum, Polygon, etc.)

Overview

CVA is the **native compliant asset** standard of the Cleanverse Compliance Protocol (CCP). It is issued **directly** on Cleanverse by qualified issuers, and every transfer is gated through **CVI (Cleanverse Verified Identity) compliance verification** and the **RuleV2** policy engine.

CVA uses RuleV2 policies to guarantee that only qualified users can hold or transfer the asset, satisfying institutional-grade KYC / AML / Travel Rule regulatory requirements.

Key Features

- **Direct issuance**: no pre-existing original ERC20 required
- Built-in **CVI compliance verification**
- Compliance policy based on **RuleV2** (Group / Sub-Group / Tier / Sub-Tier / Country Bitmap)
- Supports **Travel Rule** reporting
- Supports **Pause / Resume** and **Whitelist** operational controls
- Two integration paths: **API Launch** and **Custom Contract Template**
- Standard ERC20 + compliance hooks based on **OpenZeppelin v5**

RuleV2 Overview

> This section is a quick reference card for developers. The complete field definition lives in the contract layer `IComplianceRule.sol`.

`RuleV2` is the policy structure used by the Cleanverse compliance engine when evaluating transfer compliance. Each token holds a list `RuleV2[]` evaluated under **OR** semantics: a user is considered compliant if they match **any one** rule; the same token can register multiple rules to express differentiated access paths such as "regional exemptions", "institutional whitelists", or "high-tier express lanes".

`RuleV2` field definition:

RuleV2

```
struct RuleV2 {
    bytes2  allowedGroup;        // Allowed CVI group (0x0000 = unrestricted)
    bytes2  allowedSubGroup;     // Allowed CVI sub-group (0x0000 = unrestricted)
    uint8   minTier;             // Minimum CVI tier (0 = unrestricted; range 0-99)
    uint8   minSubTier;          // Minimum sub-tier (0 = unrestricted; range 0-99)
    uint256 poolCountryBitmap;   // Country bitmap (0 = unrestricted; bitwise AND check)
}
```

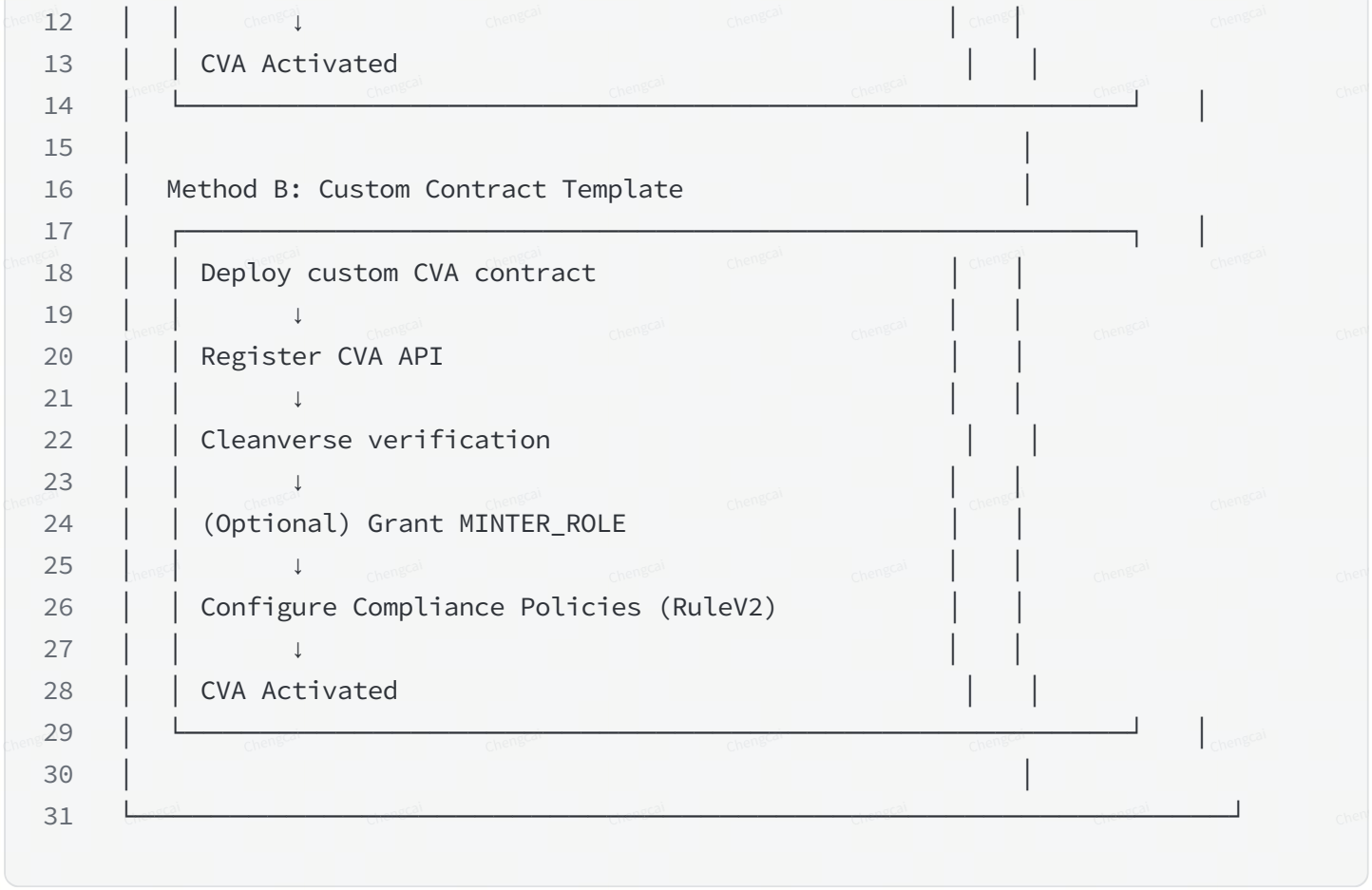
> Please read the `IComplianceRule` source file directly for field details, OR/AND semantics, and operational controls. **This document does not re-expand them.**

Integration Architecture

CVA integration is composed of the following modules:

代码块

```
┌──────────────────────────────────────────────────────────────────────┐
│                    CVA Integration Architecture                      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Method A: API Launch                                                │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ Launch CVA API                                          │    │
│  │       ↓                                                 │    │
│  │ Cleanverse verification                                  │    │
│  │       ↓                                                 │    │
│  │ (Optional) Admin Grants MINTER_ROLE                     │    │
```



CVA Issuance

CVA provides two recommended integration paths. Developers can choose **either one**:

Method	Suitable scenarios	Key difference
Method A: API Launch	Existing backend infrastructure, automated issuance, integration with existing business systems	Cleanverse deploys the contract template uniformly; policies are pushed via API
Method B: Contract template	Full control over token logic, existing mature contract infrastructure, custom business requirements	You deploy the ERC20 contract yourself; self-manage RuleV2 via *FromToken entry points

---

Method A: Launch CVA via API

Suitable scenarios

- Existing backend infrastructure
- Need automated issuance workflows
- Need integration with existing business systems

- Do not want to maintain contract code in-house

Workflow

```
Launch CVA Workflow

Call Launch CVA API
↓
Cleanverse Review
↓
 (Optional) Admin Grants MINTER_ROLE
↓
CVA Activated
```

Step 1: Call Launch CVA API

Call the Launch CVA API to submit an issuance request.

Token configuration

Field	Required	Description
chain	✓	Target network (e.g. base)
token_name	✓	CVA name
token_symbol	✓	CVA symbol
decimals	✓	Token decimals
admin_address	✓	Admin wallet address for the contract
rule	✓	**RuleV2 rule object** (see fields below)
icon	✗	CVA icon URL
callback_url	✗	Async callback URL

RuleV2 rule object (API layer)

> The API layer uses snake_case naming; field semantics align exactly with the on-chain `RuleV2`:

API field	On-chain field	Type	Allowed values	Meaning
allowed_group	allowedGroup	string	e.g. "ab", ""	User's group must equal this value; empty means
allowed_sub_group	allowedSubGroup	string	e.g. "BK", ""	User's subGroup must equal this value; empty m
min_tier	minTier	uint8	0-99	0 means unrestricted; values above 99 are rejecte
min_sub_tier	minSubTier	uint8	0-99	0 means unrestricted; values above 99 are rejecte
is_black_list	—	bool	true / false	**Deprecated field**; always pass false. Replaced by poolCountryBitmap
countries	poolCountryBitmap	string[]	ISO country code list	Converted to a country bitmap and checked via b

> **Important migration note**: The legacy `is_black_list` + `countries` array is consolidated into `poolCountryBitmap` (a 256-bit bitmap, bit positions correspond to ISO 3166-1 numeric codes) in RuleV2. If you use the new API, only pass RuleV2 fields; if the current API still exposes `countries`, pass country codes as a string array and the platform side will handle compatibility.

RuleV2 example

example

```
{
  "allowed_group": "",
  "allowed_sub_group": "",
  "min_tier": 30,
  "min_sub_tier": 0,
  "is_black_list": false,
  "countries": ["US", "SG"]
}
```

> This rule means: only requires a valid CVI with tier ⩾ 30, and allows US/SG users (a zero country bitmap means no country restriction).

Submission

1. Encrypt the `data` field using AES/CBC/PKCS5Padding
2. Base64-encode the ciphertext into the request body
3. Set the `api-id` and `X-Request-ID` headers
4. POST to `POST /api/cooperate/atoken/launch`

Step 2: Cleanverse verification

Cleanverse verifies:

- Token configuration correctness
- **RuleV2 compliance policy validity**
- Business scenario suitability
- Issuer qualifications (CVA is open only to qualified issuers)

You can poll the verification status via the Query Apply Status API.

Step 3: (Optional) Grant `MINTER_ROLE`

After verification passes, **if your business requires AccessCore or other platform contracts to mint on your behalf**, the Admin address must grant mint permission on the CVA contract to the target contract; otherwise you can skip this step and let your own `MINTER_ROLE` holder mint directly.

Prerequisites

- Verification has passed
- Admin address holds sufficient ETH for gas

Steps

1. Get the CVA contract address (available via the status query API after verification completes)
2. Get the target contract address (e.g. AccessCore, see Query Supported CVA List)
3. Call `grantRole` from the Admin wallet

ethers.js example

grant minter_role example

```
const { ethers } = require('ethers');

const aTokenAddress     = "0x...";
const minterContractAddr = "0x...";

const abi = [
  "function grantRole(bytes32 role, address account) external"
];

const MINTER_ROLE = ethers.keccak256(
  ethers.toUtf8Bytes("MINTER_ROLE")
);

const provider = new ethers.JsonRpcProvider("https://mainnet.base.org");
const wallet   = new ethers.Wallet("0x...", provider);
const contract = new ethers.Contract(aTokenAddress, abi, wallet);

const tx = await contract.grantRole(MINTER_ROLE, minterContractAddr);
```

```
19  await tx.wait();
20  console.log("Grant transaction confirmed:", tx.hash);
```

The CVA is now activated and ready for day-to-day transfers and operational management.

Method B: Issue CVA via Contract Template

Suitable scenarios

- Full control over token logic
- Custom business requirements
- Existing mature contract infrastructure
- Need a choice between upgradeable and non-upgradeable contracts

Workflow

Contract Template workflow

```
1   Deploy Custom CVA Contract
2   ↓
3   Call Register CVA API
4   ↓
5   Cleanverse Review
6   ↓
7    (Optional) Grant MINTER_ROLE
8   ↓
9   Configure Compliance Policies (RuleV2)
10  ↓
11  CVA Activated
```

Step 1: Deploy the custom contract

> Important: You only need to focus on **RuleV2** and the compliance hooks; **do not call or be aware of legacy Rule interfaces**. The contract templates below use `RuleV2` from `IComplianceRule` directly.

Contract requirements

To satisfy CCP requirements, custom contracts must:

- Specify the `policy` contract address during initialization
- Call `policy.canTransfer(...)` before transfers
- Implement `Ownable` or `AccessControl`

- Support Mint / Burn
- (Recommended) Support AccessControl

Minimal interface (RuleV2-only view)

IATokenPolicy

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @dev IComplianceRule (RuleV2-only view; developers do not need to care about the legacy Rule)
interface IComplianceRule {
    struct RuleV2 {
        bytes2  allowedGroup;
        bytes2  allowedSubGroup;
        uint8   minTier;
        uint8   minSubTier;
        uint256 poolCountryBitmap;
    }
}

interface IATokenPolicy is IComplianceRule {
    error TransferNotAllowed();

    function canTransfer(address token, address from, address to, uint256 amount)
        external view returns (bool);

    function setRuleV2(address token, RuleV2 calldata rule) external;
    function addRuleV2(address token, RuleV2 calldata rule) external;
    function removeRuleV2(address token, uint256 index)     external;
    function setRuleV2FromToken(RuleV2 calldata rule)       external;
    function addRuleV2FromToken(RuleV2 calldata rule)       external;
    function removeRuleV2FromToken(uint256 index)           external;
    function getRulesV2(address token) external view returns (RuleV2[] memory);
}
```

Contract template (Upgradeable · Recommended)

Upgradeable Contract template

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

```

```
import {ERC20Upgradeable}        from "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import {AccessControlUpgradeable} from "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";
import {Initializable}           from "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import {OwnableUpgradeable}      from "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
import {IATokenPolicy}           from "./IATokenPolicy.sol";

contract PartnerCompliantATokenV2 is
    Initializable,
    ERC20Upgradeable,
    AccessControlUpgradeable,
    OwnableUpgradeable
{
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    IATokenPolicy public policy;
    uint8 private _decimals;

    function initialize(
        string memory name_,
        string memory symbol_,
        uint8 decimals_,
        address policy_,
        address admin_
    ) external initializer {
        require(policy_ != address(0), "policy=0");
        require(admin_   != address(0), "admin=0");

        __ERC20_init(name_, symbol_);
        __AccessControl_init();
        __Ownable_init(admin_);

        policy = IATokenPolicy(policy_);
        _decimals = decimals_;
        _grantRole(DEFAULT_ADMIN_ROLE, admin_);
    }

    function decimals() public view override returns (uint8) { return _decimals; }

    function _update(address from, address to, uint256 amount) internal override {
        if (!policy.canTransfer(address(this), from, to, amount)) {
            revert IATokenPolicy.TransferNotAllowed();
        }
```

```
        super._update(from, to, amount);
    }

    function mint(address to,   uint256 amount) external onlyRole(MINTER_ROLE) { _mint(to,   amount); }
    function burn(address from, uint256 amount) external onlyRole(MINTER_ROLE) { _burn(from, amount); }

    // ── RuleV2 self-management (only DEFAULT_ADMIN_ROLE may call; routes to *FromToken) ──

    /// Replace: clear the existing RuleV2[] and set the new one
    function setRuleV2(IATokenPolicy.RuleV2 calldata rule) external onlyRole(DEFAULT_ADMIN_ROLE) {
        policy.setRuleV2FromToken(rule);
    }

    /// Append: add a RuleV2 at the end of the array (OR semantics)
    function addRuleV2(IATokenPolicy.RuleV2 calldata rule) external onlyRole(DEFAULT_ADMIN_ROLE) {
        policy.addRuleV2FromToken(rule);
    }

    /// Remove: delete a RuleV2 by index
    function removeRuleV2(uint256 index) external onlyRole(DEFAULT_ADMIN_ROLE) {
        policy.removeRuleV2FromToken(index);
    }

    /// Query the full set of current RuleV2s
    function getRulesV2() external view returns (IATokenPolicy.RuleV2[] memory) {
        return policy.getRulesV2(address(this));
    }
}
```

Contract template (Non-upgradeable)

Non-upgradeable Contract template

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC20}         from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Ownable}       from "@openzeppelin/contracts/access/Ownable.sol";
import {IATokenPolicy} from "./IATokenPolicy.sol";
```

```
7
8   contract PartnerCompliantERC20V2 is ERC20, Ownable {
9       IATokenPolicy public immutable policy;
10
11      constructor(
12          string memory name_,
13          string memory symbol_,
14          address policy_
15      ) ERC20(name_, symbol_) Ownable(msg.sender) {
16          require(policy_ != address(0), "policy=0");
17          policy = IATokenPolicy(policy_);
18      }
19
20      function _update(address from, address to, uint256 amount) internal
    override {
21          if (!policy.canTransfer(address(this), from, to, amount)) {
22              revert IATokenPolicy.TransferNotAllowed();
23          }
24          super._update(from, to, amount);
25      }
26  }
```

For non-upgradeable contracts that need to expose RuleV2 management entry points, simply copy the `setRuleV2` / `addRuleV2` / `removeRuleV2` / `getRulesV2` wrappers above.

Step 2: Call Register CVA API

After deployment, call the Register CVA API to activate compliance capabilities.

Cleanverse verifies the registration is initiated by the contract owner via `owner_signature`.

API fields

Field	Required	Description
chain	✓	Target network
atoken_address	✓	Your deployed CVA contract address
atoken_icon	✗	CVA icon
owner_signature	✓	EIP-191 personal_sign signature over lowercase(chain + atoken_address)
callback_url	✗	Async callback URL

> Note: The registration API does not take a RuleV2; after the contract is registered, you self-manage the rules via `setRuleV2` / `addRuleV2`, or append them via the Add CVA Rule API.

Step 3: Cleanverse verification

Cleanverse verifies:

- Contract implementation compliance
- Policy integration correctness
- **RuleV2** configuration compliance

Step 4: (Optional) Grant `MINTER_ROLE`

After verification passes, **if your business requires AccessCore or other platform contracts to mint on your behalf**, the Admin address must grant mint permission on the CVA contract to the target contract; otherwise you can skip this step. The flow is identical to Method A's Step 3.

Step 5: Configure RuleV2 compliance policies

After contract registration you should self-manage RuleV2 rules via `setRuleV2` / `addRuleV2` (recommended), or append them via the Add CVA Rule API. The CVA is fully operational only after rules are configured.

Full API documentation: https://docs.cleanverse.com/